In [ ]:
```
import marimo as mo
from z3 import BitVec, BitVecVal, Implies, And, Or, Not, Solver, UGE, ULT,
ULE, UDiv, LShR, simplify, sat
```

Verificação Formal de um Algoritmo de Multiplicação de Inteiros

Descrição do Problema

O presente trabalho visa a modelação e verificação formal de um programa imperativo que calcula o produto de dois inteiros sem sinal, a e b, com precisão limitada a n bits, descrito através de um *Control Flow Automaton* (CFA).

O sistema manipula as variáveis de registo x, y, z através de operações bitwise e deslocamentos (`<< 1` e `>> 1`), acumulando em z o resultado da multiplicação. Como as operações aritméticas numa ALU real ativam exceções de *overflow* e no Z3 a semântica de `BitVec` opera estritamente em aritmética modular $(\mathrm{mod}\ 2^n)$, o modelo formal inclui estados explícitos de erro (`pc = 4`) para capturar potenciais ultrapassagens de capacidade tanto na duplicação de x como na acumulação em z.

Pretende-se provar via **k-indução** e análise de **variantes**:

1. A correção funcional do invariante de acumulação ($x \cdot y + z = a \cdot b$).
2. A segurança contra *overflow* (inatingibilidade de `pc = 4`) sob pré-condições apropriadas ($a, b < 2^{n/2}$).
3. A terminação garantida do algoritmo em qualquer execução admissível.

In [ ]:
```
n_bits = 16
```

In [ ]:
```
def declare(i):
    state = {}

    state['pc'] = BitVec(f"pc_{i}", n_bits)

    state['a'] = BitVec(f"a_{i}", n_bits)
    state['b'] = BitVec(f"b_{i}", n_bits)
    state['x'] = BitVec(f"x_{i}", n_bits)
    state['y'] = BitVec(f"y_{i}", n_bits)
    state['z'] = BitVec(f"z_{i}", n_bits)
    return state
```

In [ ]:
```
def limites(state):
    # 1 << (n_bits // 2) é uma forma muito eficiente em Python de fazer
2^(n/2)
    limite = BitVecVal(1 << (n_bits // 2), n_bits)
    return And(
```

```
        ULT(state['a'], limite),
        ULT(state['b'], limite)
    )
```

Modelação dos Estados Iniciais (`init` vs `init_seguro`)

O código disponibiliza duas condições iniciais distintas para responder aos requisitos teóricos das diferentes alíneas

- **`init` (Condição Não Restrita - Alínea B):** Exige unicamente que o autómato arranque no nó inicial ($pc = 0$). Aqui, os inputs a e b são inteiramente livres e não têm restrições de magnitude. Esta formulação é usada na Alínea B para provar que a correção funcional da multiplicação em aritmética modular ($(x \cdot y) + z \equiv a \cdot b \pmod{2^n}$) é válida independentemente de eventuais saturações.
- **`init_seguro` (Condição Restrita - Alíneas C e D):** Acrescenta ao estado inicial a pré-condição exigida no enunciado ($a < 2^{n/2} \wedge b < 2^{n/2}$) através da função auxiliar `limites` . Esta restrição é indispensável para garantir a ausência de *overflow*: ao limitar os inputs à metade da capacidade de bits dos registos, assegura-se que o produto total $a \cdot b$ nunca ultrapassa $2^n - 1$, impedindo que as operações atinjam o estado de erro ($pc = 4$).

In [ ]:
```
def init(state):
    # Usado nas alíneas A e B
    return state['pc'] == BitVecVal(0, n_bits)

def init_seguro(state):
    # Usado na alínea C (com os limites)
    return And(
        init(state),
        limites(state)
    )
```

- Na versão original, a função trans usava um grande bloco Or onde cada possível transição de estado (cada And) declarava exaustivamente todas as variáveis que se mantinham inalteradas.
- O principal problema computacional era que as variáveis a e b (que representam os inputs iniciais e nunca mudam durante a execução) eram re-declaradas como inalteradas (prox['a'] == curr['a'], etc.) no interior de cada um dos ramos da execução.
- Para o Z3, isto significa que, ao explorar diferentes caminhos da máquina de estados, o solver tinha de processar e validar repetidamente esta igualdade, lidando com uma fórmula logicamente equivalente, mas estruturalmente massiva e redundante.

-
- Na nova versão, introduziu-se a função auxiliar unchanged que traz duas grandes vantagens:
- Fatorização Global: A alteração mais impactante foi a reestruturação lógica. A nova versão faz And(unchanged(curr, prox, ['a', 'b']), Or(...)). Ou seja, a igualdade de a e b foi "puxada" para fora do Or. O solver recebe agora a informação antecipada e global de que, independentemente do que aconteça no interior da máquina de estados, a e b são constantes.
- Compactação Local: Para as restantes variáveis (x, y, z), o uso de unchanged dentro de cada ramo do Or gera cláusulas mais compactas. O uso de simplify em cima desta estrutura mais limpa consegue otimizar as expressões lógicas de forma muito mais eficaz antes de as enviar para o motor de SAT/SMT.
-
- Ao remover redundâncias e declarar o comportamento de a e b globalmente, o espaço de procura do solver encolhe drasticamente:
- Para 9 bits: O Z3 passou de demorar 51 segundos (na versão antiga) para apenas 8 segundos (na versão nova).
- Escalabilidade: Com a versão ineficiente, testar 11 ou mais bits tornar-se-ia impraticável. Com a nova versão otimizada, o teste de 11 bits corre agora em 56 segundos, provando que uma correta modelação algébrica e fatorização das invariantes são tão importantes na verificação formal como a capacidade de processamento da máquina.

In [ ]:
```python
def unchanged(curr, prox, vars_list):
    return And([prox[v] == curr[v] for v in vars_list])

def trans(curr, prox):
    return simplify(And(
        unchanged(curr, prox, ['a', 'b']),
        Or(
            And( # init (0) -> skip (1)
                curr['pc'] == BitVecVal(0, n_bits),
                prox['pc'] == BitVecVal(1, n_bits),
                prox['x'] == curr['a'],
                prox['y'] == curr['b'],
                prox['z'] == BitVecVal(0, n_bits)
            ),

            And( # skip (1) -> left (2)
                curr['pc'] == BitVecVal(1, n_bits),
                curr['y'] != BitVecVal(0, n_bits),
                (curr['y'] & 1) == 0,
                prox['pc'] == BitVecVal(2, n_bits),
                unchanged(curr, prox, ['x', 'y', 'z'])
            ),

            And( # skip (1) -> right (3)
                curr['pc'] == BitVecVal(1, n_bits),
                curr['y'] != BitVecVal(0, n_bits),
```

```
            (curr['y'] & 1) == 1,
            prox['pc'] == BitVecVal(3, n_bits),
            unchanged(curr, prox, ['x', 'y', 'z'])
        ),

        And( # skip (1) -> stop (5)
            curr['pc'] == BitVecVal(1, n_bits),
            curr['y'] == BitVecVal(0, n_bits),
            prox['pc'] == BitVecVal(5, n_bits),
            unchanged(curr, prox, ['x', 'y', 'z'])
        ),

        And( # left (2) -> skip (1)
            curr['pc'] == BitVecVal(2, n_bits),
            UGE(curr['x'] + curr['x'], curr['x']),
            prox['pc'] == BitVecVal(1, n_bits),
            prox['x'] == curr['x'] << 1,
            prox['y'] == LShR(curr['y'], 1),
            unchanged(curr, prox, ['z'])
        ),

        And( # left (2) -> error (4)
            curr['pc'] == BitVecVal(2, n_bits),
            ULT(curr['x'] + curr['x'], curr['x']),
            prox['pc'] == BitVecVal(4, n_bits),
            unchanged(curr, prox, ['x', 'y', 'z'])
        ),

        And( # right (3) -> skip (1)
            curr['pc'] == BitVecVal(3, n_bits),
            UGE(curr['z'] + curr['x'], curr['z']),
            prox['pc'] == BitVecVal(1, n_bits),
            prox['y'] == curr['y'] - 1,
            prox['z'] == curr['z'] + curr['x'],
            unchanged(curr, prox, ['x'])

        ),

        And( # right (3) -> error (4)
            curr['pc'] == BitVecVal(3, n_bits),
            ULT(curr['z'] + curr['x'], curr['z']),
            prox['pc'] == BitVecVal(4, n_bits),
            unchanged(curr, prox, ['x', 'y', 'z'])
        ),

        And( # error (4) -> error (4)
            curr['pc'] == BitVecVal(4, n_bits),
            prox['pc'] == BitVecVal(4, n_bits),
            unchanged(curr, prox, ['x', 'y', 'z'])
        ),

        And( # stop (5) -> stop (5)
            curr['pc'] == BitVecVal(5, n_bits),
            prox['pc'] == BitVecVal(5, n_bits),
            unchanged(curr, prox, ['x', 'y', 'z'])
        )
```

```
        )
    ))
```

Invariante Funcional do Algoritmo (`invariante`)

A função `invariante` traduz a propriedade de acumulação essencial que preserva a correção do algoritmo de multiplicação (multiplicação binária):

$$(x \cdot y) + z = a \cdot b$$

Em termos operacionais, o acumulador z recebe progressivamente as contribuições das potências de 2 à medida que x é duplicado ($x \ll 1$) e y é decrementado ou dividido por 2 ($y \gg 1$).

A propriedade é codificada como uma implicação condicional: aplica-se estritamente quando $pc \neq 0$, dado que no ciclo inicial ($pc = 0$) os registos x, y, z ainda se encontram num estado transiente antes de serem inicializados respetivamente com $a, b, 0$.

In [ ]:
```
def invariante(state):
    return Implies(
        state['pc'] != BitVecVal(0, n_bits),
        (state['x'] * state['y']) + state['z'] == state['a'] * state['b']
        )
```

Reforço Indutivo de Segurança (`invariante_forte`)

O invariante funcional básico ($(x \cdot y) + z = a \cdot b$) é insuficiente para provar indutivamente a inatingibilidade do estado de erro ($pc \neq 4$) na Alínea C. Na verificação por k-indução, o *solver* precisa de descartar estados inalcançáveis no passo indutivo.

A função `invariante_forte` fortalece a hipótese indutiva através da conjunção dos seguintes lemas:

- **Propriedade de Segurança:** Imposição de que o estado de erro nunca ocorre ($pc \neq 4$).
- **Limites de Inputs e Acumulação:** Garantia de que os limites persistem e que os registos intermédios respeitam as cotas superiores $y \leq b$ e $z \leq a \cdot b$.
- **Cota Hiperbólica contra Overflow:** Para $y \neq 0$, restringe-se $x < \lfloor (2^n - 1)/y \rfloor$, assegurando matematicamente que o produto $x \cdot y$ não satura a capacidade do registo de n bits.
- **Sincronização com o Program Counter:** Condições estruturais de paridade que amarram o estado interno ao valor de pc ($pc = 2 \implies y$ é par, $pc = 3 \implies y$ é ímpar e $pc = 5 \implies y = 0$).

In [ ]:
```
def invariante_forte(state):
    return simplify(And(
        state['pc'] != BitVecVal(4, n_bits),

        limites(state),

        Implies(
            state['pc'] != BitVecVal(0, n_bits),
            And(
                invariante(state),
                ULE(state['y'], state['b']),
                ULE(state['z'], state['a'] * state['b']),
                Implies(
                    state['y'] != BitVecVal(0, n_bits),
                    ULT(state['x'], UDiv(BitVecVal(2**n_bits - 1, n_bits),
state['y']))
                    # cria um vetor com tudo a 111...11, dividimos o valor
por y
                )
            )
        ),
        Implies(
            state['pc'] == BitVecVal(2, n_bits),
            And(
                state['y'] != BitVecVal(0, n_bits),
                (state['y'] & 1) == 0
            )
        ),
        Implies(
            state['pc'] == BitVecVal(3, n_bits),
            And(
                state['y'] != BitVecVal(0, n_bits),
                (state['y'] & 1) == 1
            )
        ),
        Implies(
            state['pc'] == BitVecVal(5, n_bits),
            state['y'] == BitVecVal(0, n_bits)
        )
    )
    )
```

Função Variante e Prova de Terminação (`variante`)

A função de variante estabelece uma medida de progresso sobre o espaço de estados do autómato para demonstrar que o programa termina sem incorrer em ciclos infinitos:

$$V(s) = y$$

O registo y atua como o contador estrito do ciclo de multiplicação:

1. A cada passagem pelo ramo de redução par ($pc = 2$), y sofre um deslocamento lógico à direita ($y \gg 1$), diminuindo estritamente o seu valor.
2. A cada passagem pelo ramo ímpar ($pc = 3$), y é decrementado numa unidade ($y - 1$).

Como y é um vetor de bits interpretado como um número inteiro sem sinal ($\mathbb{N}$ limitado a n bits), o seu domínio é bem-ordenado e tem como limite inferior o valor 0. Ao garantir que $V(s)$ decresce estritamente a cada par de transições ($k = 2$) ou atinge 0, prova-se formalmente que o algoritmo atinge o nó de paragem ($pc = 5$) em tempo finito.

In [ ]:
```
def variante(state):
    return state['y']
```

In [ ]:
```
def pre_condicao_lookahead(state):
    return And(
        state['pc'] != BitVecVal(0, n_bits),
        invariante_forte(state)
    )
```

Avaliação do Solver e Heurísticas: `Solver()` vs `QF_BV`

Testou-se empiricamente o uso de `SolverFor("QF_BV")` em comparação com o `Solver()` genérico do Z3. Como a configuração padrão do Z3 apresentou um desempenho substancialmente superior na resolução das fórmulas, manteve-se o `Solver()` genérico.

In [ ]:
```
def kinduction_always(declare, init, trans, inv, k):
    solver = Solver()

    states = [declare(i) for i in range(k)]

    base = [init(states[0])]
    for i in range(k - 1):
        base.append(trans(states[i], states[i+1]))

    caso_base = simplify(And(And(*base), Or(*[Not(inv(s)) for s in
states])))

    solver.add(caso_base)
    if solver.check() == sat:
        print("Falha caso base")
        print("CE:", solver.model())
        return False

    solver.reset()

    states = [declare(i) for i in range(k + 1)]

    inductive = []
```

```
    for i in range(k):
        inductive.append(trans(states[i], states[i+1]))

    inductive.append(And(*[inv(states[i]) for i in range(k)]))

    inductive.append(Not(inv(states[k])))

    step_case = simplify(And(*inductive))

    solver.add(step_case)
    if solver.check() == sat:
        print("Falha passo indutivo")
        print("CE:", solver.model())
        return False

    print("Propriedade válida por k-indução")
    return True
```

In [ ]:
```
def k_lookahead(declare, pre_cond, trans, V, k):
    solver = Solver()

    states = [declare(i) for i in range(k + 1)]

    solver.add(pre_cond(states[0]))

    transitions = [trans(states[i], states[i+1]) for i in range(k)]
    chain = And(*transitions)

    decreases = Or(
        ULT(V(states[-1]), V(states[0])),
        V(states[-1]) == BitVecVal(0, n_bits)
    )

    solver.add(simplify(And(chain, Not(decreases))))

    if solver.check() == sat:
        print(f"Falha {k} transições sem decrescer")
        print("CE:", solver.model())
        return False

    print("Variante válido")
    return True
```

Análise da Discrepância de Tempos entre Alíneas

A execução dos testes revela que a verificação indutiva da **Alínea B** consome a larga maioria do tempo de computação. Este comportamento decorre de dois fatores principais:

- **Multiplicação Não Linear em Vetores de Bits:** A propriedade $(x \cdot y) + z = a \cdot b$ exige que o solver resolva equações com produtos de variáveis simbólicas de n bits. A síntese de circuitos multiplicadores via *bit-blasting* gera um número de variáveis proposicionais e restrições que cresce quadraticamente com n ($O(n^2)$),

tornando a resolução em lógica SAT muito exigente. Em contrapartida, as alíneas C e D apoiam-se maioritariamente em adições, deslocamentos lógicos e comparações de ordem, operações cuja tradução lógica é quase linear.

- **Espaço de Procura sem Limites nos Inputs:** Na Alínea B, o estado inicial não impõe qualquer limite aos valores de a e b, obrigando o Z3 a explorar todo o espaço de $2^n \times 2^n$ combinações e a tratar a aritmética modular mesmo sob eventuais situações de saturação.

O Papel do Reforço do Invariante (Alínea C)

A verificação por indução padrão ($k = 1$ ou $k = 2$) da propriedade de segurança pura ($\mathrm{pc} \neq 4$) falha se considerarmos o invariante fraco isolado. O solver consegue gerar contra-exemplos ao assumir estados iniciais estruturalmente impossíveis na execução real (ex.: um valor de x já excessivamente grande combinado com um y ímpar que conduz de imediato a um *overflow*).

A construção de `invariante_forte` resolve este problema ao adicionar lemas estruturais auxiliares:

- **Limites de Crescimento:** As condições $y \leq b$, $z \leq a \cdot b$ e a restrição hiperbólica $x < (2^n - 1)/y$ garantem que a grandeza acumulada nunca excede a capacidade do registo de n bits.
- **Sincronização com o Program Counter:** Condições que amarram a paridade e nulidade de y a cada modo do autómato ($pc = 2 \implies y$ é par, $pc = 3 \implies y$ é $\mathrm{ímpar}$, $pc = 5 \implies y = 0$).

Ao restringir o conjunto de estados candidatos aos estados estritamente acessíveis, o reforço poda a árvore de pesquisa do solver, tornando a prova da Alínea C muito mais célere e indutivamente robusta.

Garantia de Terminação (Alínea D)

O método de k-lookahead com $k = 2$ prova que em qualquer par de transições válidas a partir de um estado acessível, o variante estritamente decresce até atingir o mínimo no domínio bem ordenado dos inteiros sem sinal ($\mathbb{N}$ limitado a n bits). Isto garante que o sistema atinge o nó de paragem sem ciclos infinitos.

In [ ]:
```python
import time

print("\n--- ALÍNEA B ---")
print("A testar (x*y+z == a*b) por k-indução.")
t0 = time.time()
kinduction_always(declare, init, trans, invariante, 2)
print(f"Demorou: {time.time() - t0:.2f} segundos")

print("\n--- ALÍNEA C ---")
```

```
print("A testar se o estado de erro (pc=4) é inatingível por k-indução.")
t0 = time.time()
kinduction_always(declare, init_seguro, trans, invariante_forte, 2)
print(f"Demorou: {time.time() - t0:.2f} segundos")

print("\n--- ALÍNEA D ---")
print("A testar se o variante 'y' decresce.")
t0 = time.time()
k_lookahead(declare, pre_condicao_lookahead, trans, variante, 2)
print(f"Demorou: {time.time() - t0:.2f} segundos")
```

Registo da interação com a LLM: `https://share.gemini.google/U8q87XIOU1al`